ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN

Môn: NGÔN NGỮ LẬP TRÌNH C#

Tên đề tài: CLASS LIBRARY & UNIT TEST

Giảng viên hướng dẫn: Đỗ Như Tài

Sinh viên thực hiện: Huỳnh Công Tính

MSSV: 3124411315

Lớp: DCT124C6

TP. HỒ CHÍ MINH, THÁNG 9 NĂM 2026

CHỦ ĐỀ BUỔI 02: CLASS LIBRARY & UNIT TEST

Mục lục nội dung

Phần	Nội dung
1	Thiết kế lớp cơ bản: Bài 1.1 đến 1.5
2	Thiết kế lớp nâng cao: Bài 2.1 đến 2.7
3	Interface, delegate, kế thừa và đa hình: Bài 3.1 đến 3.6
4	Kết luận và hướng dẫn chạy Unit Test

Mục tiêu báo cáo

Báo cáo trình bày quá trình phân tích từng đề bài, cách lựa chọn lớp và phương thức, công thức xử lý, kết quả chạy minh họa và các nguyên lý lập trình hướng đối tượng được áp dụng. Mã nguồn đi kèm được xây dựng trên .NET 10 và kiểm thử bằng xUnit.

Quy ước chung

Trong mã nguồn, tên lớp và phương thức được đặt bằng tiếng Việt không dấu để thuận tiện khi nhập liệu. Các giá trị không hợp lệ được kiểm tra trong property hoặc constructor. Mỗi bài có năm Unit Test, đại diện cho trường hợp đúng, giá trị biên và các trường hợp lỗi quan trọng.

Phần 1 Thiết kế lớp cơ bản

1.1 Tính tuổi sinh viên

Phân tích đề bài

Nhập họ tên và năm sinh của một sinh viên, sau đó tính tuổi theo năm hiện tại.

Hướng thiết kế và xử lý

Lớp SinhVien chứa HoTen và NamSinh.

Các property kiểm tra họ tên không được rỗng và năm sinh không được vượt quá năm hiện tại.

Tuổi được tính bằng DateTime.Now.Year trừ NamSinh.

Công thức hoặc logic chính:

Tuoi = NamHienTai – NamSinh

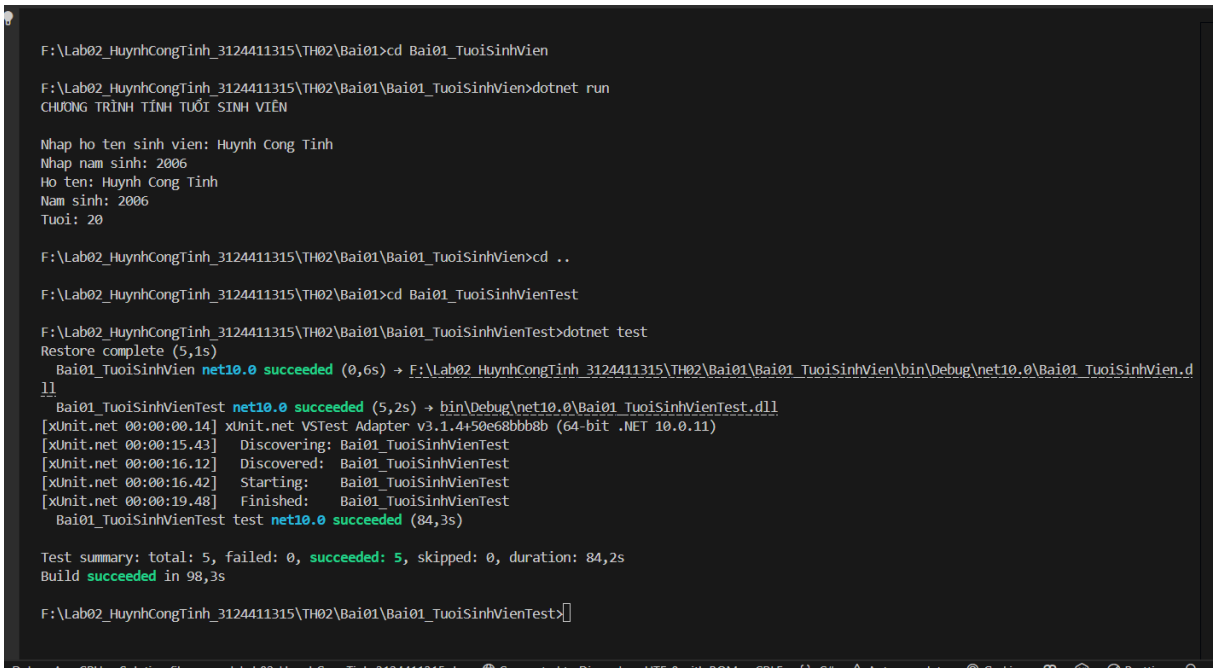
Trước khi tính tuổi, chương trình kiểm tra dữ liệu đầu vào theo các điều kiện:

- Họ tên không được để trống hoặc chỉ chứa khoảng trắng.
- Năm sinh phải là một số nguyên dương.
- Năm sinh không được lớn hơn năm hiện tại.
- Nếu dữ liệu không hợp lệ, chương trình thông báo lỗi và yêu cầu người dùng nhập lại.

Trình tự xử lý chính của chương trình:

1. Tạo một đối tượng thuộc lớp SinhVien.
2. Gọi phương thức Nhap() để nhập họ tên và năm sinh.
3. Kiểm tra tính hợp lệ của họ tên thông qua property HoTen.
4. Kiểm tra tính hợp lệ của năm sinh thông qua property NamSinh.
5. Gọi phương thức TinhTuoi() để tính tuổi.
6. Gọi phương thức Xuat() để hiển thị thông tin sinh viên và tuổi vừa tính.

Kết quả minh họa:



Giải thích cách làm

Bài này viết theo hướng đối tượng: mọi dữ liệu và thao tác liên quan đến sinh viên đều nằm trong lớp SinhVien.

Lớp có hai field private là hoTen và namSinh. Bên ngoài chỉ đọc và gán được thông qua hai property HoTen và NamSinh.

Property HoTen dùng string.IsNullOrWhiteSpace() để kiểm tra. Họ tên là null, rỗng hoặc toàn khoảng trắng thì ném ArgumentException; hợp lệ thì Trim() bỏ khoảng trắng ở hai đầu.

Property NamSinh so với DateTime.Now.Year: năm sinh phải lớn hơn 0 và không vượt quá năm hiện tại, nên nhập năm âm, bằng 0 hay năm tương lai đều bị chặn.

Nhap() gọi lần lượt hai hàm nhập họ tên và năm sinh. Với năm sinh, int.TryParse() cho biết dữ liệu có phải số nguyên hay không; nhập chữ thì báo lỗi và cho nhập lại chứ chương trình không bị dừng.

Khi dữ liệu đã hợp lệ, TinhTuoi() lấy năm hiện tại trừ năm sinh, còn Xuat() in kết quả cùng họ tên và năm sinh.

1.2 Lớp Point

Phân tích đề bài

Biểu diễn điểm trong mặt phẳng, hỗ trợ nhập xuất, ToString, các toán tử cộng, trừ, lấy điểm đối, tính khoảng cách và tìm trung điểm bằng cả phương thức thành viên lẫn phương thức tĩnh.

Hướng thiết kế và xử lý

Point có hai field x, y và hai property X, Y.

Toán tử + và - tạo đối tượng Point mới.

Khoảng cách được tính bằng căn bậc hai của tổng bình phương độ lệch tọa độ.

Mỗi tọa độ của trung điểm bằng trung bình cộng của hai tọa độ tương ứng.

Công thức hoặc logic chính

AB = sqrt((x2-x1)^2 + (y2-y1)^2)
I = ((x1+x2)/2, (y1+y2)/2)

Trình tự xử lý chính

1. Tạo hai đối tượng thuộc lớp Point.
2. Nhập hoành độ và tung độ cho từng điểm.
3. Xuất hai điểm bằng phương thức Output() hoặc ToString().
4. Thực hiện phép cộng hai điểm bằng toán tử +.
5. Thực hiện phép trừ hai điểm bằng toán tử -.
6. Lấy điểm đối bằng toán tử - một ngôi.
7. Tính khoảng cách bằng phương thức thành viên và phương thức tĩnh.
8. Tìm trung điểm bằng phương thức thành viên và phương thức tĩnh.
9. Hiển thị các kết quả ra màn hình.

Kết quả minh họa

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai01_TuoiSinhVienTest>cd ..

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01>cd Bai02_Point

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai02_Point>dotnet run
CHUONG TRINH XU LY DIEM

Nhap diem A
Nhap toa do x:  2
Nhap toa do y: 3

Nhap diem B
Nhap toa do x: 1
Nhap toa do y: 2

KET QUA
A = (2, 3)
B = (1, 2)
A + B = (3, 5)
A - B = (1, 1)
-A = (-2, -3)
Khoang cach A den B bang phuong thuc thanh vien: 1,41
Khoang cach A den B bang phuong thuc tinh: 1,41
Trung diem bang phuong thuc thanh vien: (1,5, 2,5)
Trung diem bang phuong thuc tinh: (1,5, 2,5)
```

Kết quả sau khi test

```
Trung diem bang phuong thuc tinh: (1,5, 2,5)

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai02_Point>
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai02_Point>cd ..

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01>cd Bai02_PointTest
The system cannot find the path specified.

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01>cd Bai02.PointTest

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai02.PointTest>dotnet test
Restore complete (1,0s)
  Bai02_Point net10.0 succeeded (0,2s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai02_Point\bin\Debug\net10.0\Bai02_Point.dll
  Bai02.PointTest net10.0 succeeded (1,1s) → bin\Debug\net10.0\Bai02.PointTest.dll
[xUnit.net 00:00:00.02] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:01.47]   Discovering: Bai02.PointTest
[xUnit.net 00:00:01.55]   Discovered:  Bai02.PointTest
[xUnit.net 00:00:01.59]   Starting:    Bai02.PointTest
[xUnit.net 00:00:01.97]   Finished:    Bai02.PointTest
  Bai02.PointTest test net10.0 succeeded (9,2s)

Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 9,1s
Build succeeded in 12,4s
```

Giải thích cách làm

Lớp Point biểu diễn một điểm trong mặt phẳng hai chiều, gồm hai field private x, y (hoành độ, tung độ) và hai property X, Y để đọc hoặc gán tọa độ.

Lớp cung cấp ba constructor. Constructor mặc định tạo điểm gốc tọa độ O(0,0)O(0,0). Constructor có tham số cho phép tạo một điểm với tọa độ xác định. Constructor sao chép nhận vào một đối tượng Point đã tồn tại và tạo một điểm mới có cùng tọa độ với điểm đó.

Input() dùng double.TryParse() để kiểm tra dữ liệu nhập. Nhập chữ hoặc dữ liệu không phải số thực thì báo lỗi và nhập lại.

ToString() được ghi đè để trả về chuỗi dạng (x, y), Output() chỉ việc gọi ToString() nên không phải viết lại phần định dạng.

Ba toán tử +, - hai ngôi và - một ngôi đều được nạp chồng, cho phép viết a + b, a - b, -a trực tiếp trên các đối tượng Point. Mỗi phép toán tạo một Point mới nên hai điểm ban đầu giữ nguyên.

Trước khi thực hiện phép toán, chương trình dùng:
ArgumentNullException.ThrowIfNull(point);

Lệnh này kiểm tra đối số có bị null hay không. Nếu null thì ném ngoại lệ ngay, thay vì để chương trình tính tiếp rồi lỗi ở chỗ khác, khó tìm nguyên nhân.

Khoảng cách và trung điểm đều được xây dựng theo hai hình thức:

- Phương thức thành viên được gọi từ một đối tượng cụ thể, chẳng hạn a.KhoangCachDen(b).
- Phương thức tĩnh được gọi thông qua tên lớp, chẳng hạn Point.KhoangCach(a, b).

Hai cách gọi dùng chung một công thức nên kết quả như nhau, chỉ khác cách đọc: phương thức thành viên là "điểm A tính khoảng cách đến điểm B", phương thức tĩnh là "lớp Point tính khoảng cách giữa A và B".

1.3 Lớp Person

Phân tích đề bài

Quản lý id, họ tên, năm sinh (Yob) và năm mất (Yod); xác định một người còn sống hay đã mất.

Hướng thiết kế và xử lý

Yod bằng 0 nghĩa là người đó còn sống.

Nếu Yod khác 0 thì Yod phải không nhỏ hơn Yob và không lớn hơn năm hiện tại.

Constructor sao chép tạo một đối tượng mới có cùng dữ liệu.

Công thức hoặc logic chính

IsLiving() = (Yod == 0)

Kết quả minh họa

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01>cd Bai03_Person

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai03_Person>dotnet run
CHUONG TRINH QUAN LY THONG TIN MOT NGUOI

Nhap id: A01
Nhap ho ten: Huynh Cong Tinh
Nhap nam sinh: 2006
Nhap nam mat, nhap 0 neu con song: 0

THONG TIN DA NHAP
Id: A01
Ho ten: Huynh Cong Tinh
Nam sinh: 2006
Trang thai: Con song
Huynh Cong Tinh hien tai con song.

THONG TIN BAN SAO
Id: A01
Ho ten: Huynh Cong Tinh
Nam sinh: 2006
Trang thai: Con song

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai03_Person>cd ..
```

Kết quả sau khi test

```
Nam sinh: 2006
Trang thai: Con song

                                                15\TH02\Bai01\Bai03_Person>cd ..
Focus folder in explorer (ctrl + click)
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01>cd Bai03_PersonTest

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai03_PersonTest>dotnet test
Restore complete (1,2s)
  Bai03_Person net10.0 succeeded (0,3s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai03_Person\bin\Debug\net10.0\Bai03_Person.dll
  Bai03_PersonTest net10.0 succeeded (1,0s) → bin\Debug\net10.0\Bai03_PersonTest.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:01.58]   Discovering: Bai03_PersonTest
[xUnit.net 00:00:01.81]   Discovered:  Bai03_PersonTest
[xUnit.net 00:00:01.97]   Starting:    Bai03_PersonTest
[xUnit.net 00:00:03.48]   Finished:    Bai03_PersonTest
  Bai03_PersonTest test net10.0 succeeded (12,8s)

Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 12,7s
Build succeeded in 16,1s

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai03_PersonTest>
Debug Any CPU  Solution file opened: Lab02_HuynhCongTinh_3124411315.sln  Connected to Discord  UTF-8 with BOM  CRLF  C#  Autocomplete  Go Live  Prettier
```

Giải thích cách làm

Các property bảo đảm đối tượng luôn ở trạng thái hợp lệ. Khi khởi tạo bằng constructor có tham số, năm sinh được gán trước năm mất để việc kiểm tra năm mất so với năm sinh diễn ra đúng.

1.4 Lớp Phân số

Phân tích đề bài

Xây dựng lớp phân số có constructor mặc định, constructor sao chép, phương thức ToString và nạp chồng các toán tử số học, so sánh.

Hướng thiết kế và xử lý

Mẫu số luôn khác 0.

Phân số được rút gọn bằng thuật toán Euclid.

Dấu âm được đưa lên tử số.

Ghi đè Equals và GetHashCode để phù hợp với toán tử bằng.

Công thức hoặc logic chính

a/b + c/d = (ad+bc)/bd
a/b : c/d = ad/bc

Trình tự xử lý chính

1. Tạo hai đối tượng thuộc lớp PhanSo.
2. Nhập tử số và mẫu số của từng phân số.
3. Kiểm tra mẫu số có khác 0 hay không.
4. Rút gọn và chuẩn hóa từng phân số.
5. Thực hiện các phép cộng, trừ, nhân và chia.
6. Mỗi phép toán tạo ra một đối tượng PhanSo mới.
7. Constructor của đối tượng mới tiếp tục rút gọn kết quả.
8. Xuất các phân số và kết quả ra màn hình.

Kết quả minh họa

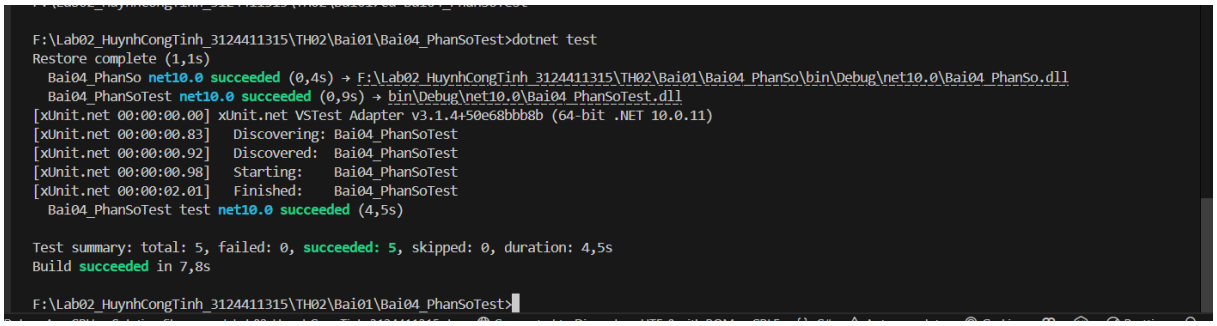
```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai04_PhanSo>dotnet run
CHUONG TRINH XU LY PHAN SO

Nhap phan so A
Nhap tu so: 4
Nhap mau so: 5

Nhap phan so B
Nhap tu so: 2
Nhap mau so: 3

KET QUA
A = 4/5
B = 2/3
+A = 4/5
-A = -4/5
A + B = 22/15
A - B = 2/15
A * B = 8/15
A / B = 6/5
A > B  : True
A < B  : False
A >= B : True
A <= B : False
A == B : False
A != B : True
```

Kết quả sau khi test:



Giải thích cách làm

Lớp PhanSo gói tử số, mẫu số và các thao tác trên phân số. Hai field tuSo, mauSo để private.

Property MauSo kiểm tra mẫu số, nhập 0 thì ném ngoại lệ vì phép chia cho 0 không xác định. Khi nhập từ bàn phím có thể dùng int.TryParse() trong vòng lặp để xử lý trường hợp nhập chữ hoặc sai định dạng.

Phân số được rút gọn ngay khi tạo: tìm ước chung lớn nhất của trị tuyệt đối tử số và mẫu số rồi chia cả hai cho số đó, nên kết quả lúc nào cũng ở dạng tối giản.

Dấu âm luôn được đưa lên tử số, ví dụ 2/-3 được lưu thành -2/3. Cách biểu diễn thống nhất này làm việc xuất và so sánh phân số đơn giản hơn.

Bốn toán tử +, -, *, / được nạp chồng để viết phép tính giữa các phân số như với số thông thường:

PhanSo tong = a + b;

PhanSo hieu = a - b;

PhanSo tich = a * b;

PhanSo thuong = a / b;

Mỗi phép toán trả về một PhanSo mới nên hai phân số ban đầu không đổi. Constructor của đối tượng mới tự kiểm tra mẫu số, đưa dấu về tử số và rút gọn.

Riêng phép chia phải kiểm tra phân số chia: nếu tử số của nó bằng 0 thì giá trị bằng 0, không chia được, và chương trình ném DivideByZeroException.

Phương thức ToString() được ghi đè để hiển thị phân số theo dạng tuSo/mauSo. Nếu mẫu số bằng 1, chương trình chỉ hiển thị tử số để kết quả tự nhiên hơn.

```
public override string ToString()
{
   if (MauSo == 1)
   {
      return TuSo.ToString();
   }

   return $"{TuSo}/{MauSo}";
}
```

1.5 Lớp Đơn thức

Phân tích đề bài

Biểu diễn P(x)=a.x^n, tính giá trị tại x và tạo đơn thức đạo hàm.

Hướng thiết kế và xử lý

Hệ số là số thực, số mũ là số nguyên không âm.

Phương thức TinhGiaTri sử dụng Math.Pow.

Phương thức DaoHam trả về một đối tượng DonThuc mới.

Công thức hoặc logic chính

P(x)=a.x^n

P'(x)=a.n.x^(n-1)

Trình tự xử lý chính

1. Tạo hai đối tượng thuộc lớp DonThuc.
2. Nhập hệ số và số mũ cho từng đơn thức.
3. Kiểm tra số mũ có hợp lệ hay không.
4. Xuất hai đơn thức bằng phương thức ToString().
5. Nhập một giá trị xx và tính giá trị của đơn thức tại xx.
6. Tính đạo hàm của từng đơn thức.
7. Kiểm tra số mũ trước khi cộng hoặc trừ.
8. Thực hiện phép nhân hai đơn thức.
9. Kiểm tra hệ số của số chia trước khi thực hiện phép chia.
10. Xuất toàn bộ kết quả ra màn hình.

Kết quả minh họa

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai05_DonThuc>dotnet run
CHUONG TRINH XU LY DON THUC

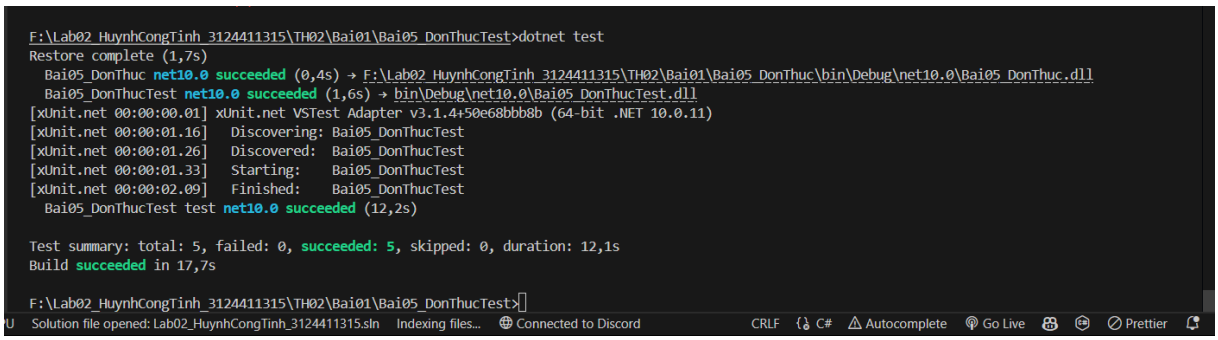
Nhap he so a: 2
Nhap so mu n: 4

Don thuc P(x) = 2x^4
Nhap gia tri x: 2

KET QUA
P(x) = 2x^4
P(2) = 32
P'(x) = 8x^3
P'(2) = 64

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai05_DonThuc>cd ..
```

Kết quả sau khi test



Giải thích cách làm

Lớp DonThuc gồm hai field private là heSo và soMu, truy cập qua property HeSo và SoMu.

Property SoMu chỉ nhận số mũ lớn hơn hoặc bằng 0 vì bài chỉ xét đơn thức của đa thức. Nếu bỏ điều kiện này để cho phép số mũ âm thì kết quả không còn là đa thức.

Phương thức TinhGiaTri() nhận một giá trị xx, sử dụng Math.Pow() để tính xnx^n, sau đó nhân với hệ số. Phương thức này chỉ trả về kết quả và không làm thay đổi đơn thức.

DaoHam() làm theo quy tắc đạo hàm lũy thừa: hệ số mới bằng hệ số cũ nhân số mũ, số mũ mới giảm 1. Đơn thức có số mũ 0 là hằng số nên đạo hàm bằng 0.

Cộng, trừ chỉ làm được khi hai đơn thức cùng số mũ: giữ phần biến và cộng hoặc trừ hệ số. Khác số mũ thì kết quả có hai hạng tử, đã là đa thức nên không trả về DonThuc được.

Nhân thì không cần cùng bậc: nhân hai hệ số, cộng hai số mũ. Chia thì chia hai hệ số, lấy số mũ thứ nhất trừ số mũ thứ hai, và phải kiểm tra hệ số của đơn thức chia khác 0 trước.

Các phép toán đều tạo DonThuc mới, hai toán hạng ban đầu không đổi.

ToString() xử lý riêng các trường hợp hệ số 0, 1, -1 và số mũ 0, 1 để in x^2 thay vì 1x^2, 3x thay vì 3x^1.

Phần 2 Thiết kế lớp nâng cao

2.1 Lớp ArrayPoint

Phân tích đề bài

Dùng ArrayList để lưu nhiều đối tượng Point và cung cấp indexer để truy cập điểm thứ i.

Hướng thiết kế và xử lý

ArrayList lưu dữ liệu dưới dạng object nên cần ép kiểu về Point.

Indexer hỗ trợ cú pháp danhSach[i].

Constructor sao chép thực hiện sao chép sâu, tạo bản sao riêng cho từng Point.

Công thức hoặc logic chính

public Point this[int index]

Trình tự xử lý chính

1. Nhập số lượng điểm cần quản lý.
2. Kiểm tra số lượng phải là số nguyên không âm.
3. Tạo đối tượng ArrayPoint với số lượng đã nhập.
4. Khởi tạo các đối tượng Point trong mảng.
5. Gọi Input() để nhập tọa độ từng điểm.
6. Gọi Output() để xuất toàn bộ danh sách.
7. Duyệt mảng để tìm điểm xa gốc tọa độ nhất.
8. Tính khoảng cách từ điểm đó đến gốc tọa độ.
9. Hiển thị điểm xa nhất và khoảng cách tương ứng.

Kết quả minh họa

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai01_ArrayPoint>dotnet run
CHUONG TRINH QUAN LY DANH SACH DIEM

Nhap so luong diem: 2

Nhap diem thu 1:
Nhap toa do x: 2
Nhap toa do y: 3

Nhap diem thu 2:
Nhap toa do x: 4
Nhap toa do y: 2

KET QUA
DANH SACH DIEM
Diem thu 1: (2, 3)
Diem thu 2: (4, 2)

Diem dau tien: (2, 3)

SAU KHI THEM DIEM (10, 20)
DANH SACH DIEM
Diem thu 1: (2, 3)
Diem thu 2: (4, 2)
Diem thu 3: (10, 20)

SAU KHI THAY DOI DIEM DAU TIEN
DANH SACH DIEM
Diem thu 1: (100, 200)
Diem thu 2: (4, 2)
Diem thu 3: (10, 20)
```

Kết quả sau khi test

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai01_ArrayPoint>cd ..

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02>cd Bai01_ArrayPointTest

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai01_ArrayPointTest>dotnet test
Restore complete (1,3s)
  Bai02_Point net10.0 succeeded (0,4s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai02_Point\bin\Debug\net10.0\Bai02_Point.dll
  Bai01_ArrayPoint net10.0 succeeded (0,3s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai01_ArrayPoint\bin\Debug\net10.0\Bai01_ArrayPoint.dll
  Bai01_ArrayPointTest net10.0 succeeded (0,9s) → bin\Debug\net10.0\Bai01_ArrayPointTest.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:00.93]   Discovering: Bai01_ArrayPointTest
[xUnit.net 00:00:01.02]   Discovered:  Bai01_ArrayPointTest
[xUnit.net 00:00:01.07]   Starting:    Bai01_ArrayPointTest
[xUnit.net 00:00:01.49]   Finished:    Bai01_ArrayPointTest
  Bai01_ArrayPointTest test net10.0 succeeded (7,1s)

Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 7,1s
Build succeeded in 11,7s

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai01_ArrayPointTest>
Solution file opened: Lab02_HuynhCongTinh_3124411315.sln  Indexing files...  Connected to Discord  CRLF  C#  Autocomplete  Go Live  Prettier
```

Giải thích cách làm

ArrayPoint không tự tính toán gì trên điểm mà dùng lại lớp Point ở bài 1.2, đó là tái sử dụng lớp.

Mảng danhSachDiem phải được khởi tạo từng phần tử bằng new Point(). Nếu chỉ tạo mảng thì mỗi vị trí đều là null, và gọi Input() trên phần tử đó sẽ gây NullReferenceException.

Input() dùng vòng lặp nhập từng điểm. Việc nhập và kiểm tra tọa độ do chính đối tượng Point đảm nhiệm, ArrayPoint chỉ điều phối thứ tự.

Output() cũng duyệt mảng và gọi ToString() của Point, nên định dạng điểm thống nhất và không phải lặp lại code.

Nhờ indexer, ArrayPoint dùng được như một mảng. Chỉ số được kiểm tra trước khi truy cập nên lỗi báo sớm và rõ ràng.

Tìm điểm xa gốc nhất không cần sắp xếp: chỉ duyệt một lần và giữ lại điểm có khoảng cách lớn nhất tính đến lúc đó. Cách này đơn giản và nhanh hơn sắp xếp.

Khi so sánh khoảng cách, dùng x^2 + y^2 thay cho sqrt(x^2 + y^2). Căn bậc hai không làm đổi thứ tự của các số không âm, nên điểm có bình phương khoảng cách lớn nhất cũng là điểm xa nhất.

2.2 Lớp PersonList

Phân tích đề bài

Quản lý nhiều đối tượng Person, cho phép thêm người và lọc ra những người còn sống.

Hướng thiết kế và xử lý

PersonList chứa List<Person>.

Phương thức LivingPeople duyệt danh sách và kiểm tra IsLiving.

Kết quả trả về dùng bản sao của Person để độc lập với danh sách gốc.

Công thức hoặc logic chính

Nếu chỉ lưu năm sinh:

Tuổi = Năm hiện tại – Năm sinh

Nếu lưu đầy đủ ngày sinh, tuổi chính xác được tính bằng cách lấy hiệu hai năm và trừ thêm 1 nếu trong năm hiện tại người đó chưa đến ngày sinh.

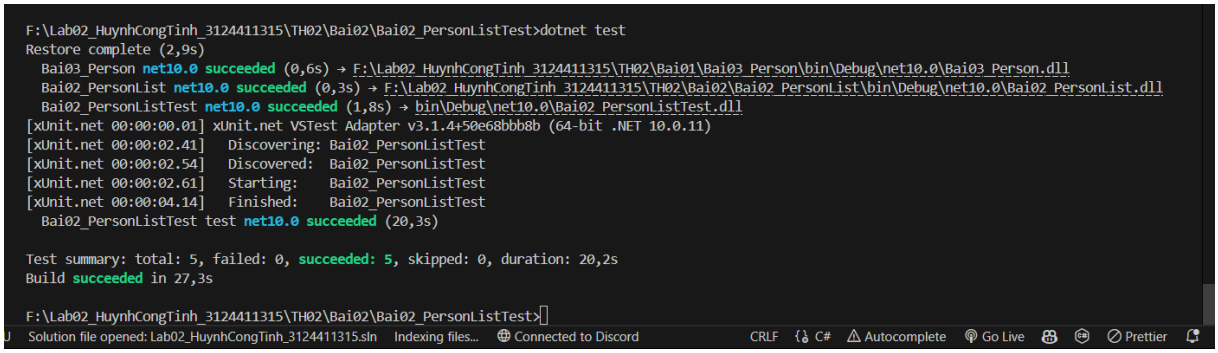
Để tìm người lớn tuổi nhất:

1. Kiểm tra danh sách không rỗng.
2. Giả sử phần tử đầu tiên là người lớn tuổi nhất.
3. Duyệt từ phần tử thứ hai.
4. Người có ngày sinh sớm hơn là người lớn tuổi hơn.
5. Cập nhật kết quả khi tìm thấy người có ngày sinh nhỏ hơn.

Kết quả minh họa



Kết quả sau khi test



Giải thích cách làm

Person quản lý dữ liệu của một người, còn PersonList chỉ quản lý tập hợp các Person và không viết lại phần nhập, xuất hay tính tuổi.

Khi tạo mảng đối tượng phải khởi tạo từng phần tử bằng new Person(), nếu không sẽ gặp NullReferenceException. Có thể truy cập phần tử qua indexer, ví dụ personList[index].

Tìm người lớn tuổi nhất chỉ cần duyệt một lần. Nếu lưu đầy đủ ngày sinh thì so sánh trực tiếp hai giá trị DateTime: ngày sinh nhỏ hơn là người sinh sớm hơn, tức lớn tuổi hơn.

2.3 Mảng một chiều

Phân tích đề bài

Xây dựng lớp quản lý mảng số nguyên một chiều và thực hiện các thao tác:

- Nhập và xuất mảng.
- Tính tổng các phần tử.
- Tìm giá trị lớn nhất, nhỏ nhất.
- Đếm số phần tử chẵn hoặc lẻ.
- Tìm kiếm một giá trị.
- Sắp xếp mảng nếu đề bài yêu cầu.

Hướng thiết kế và xử lý

Dữ liệu lưu bằng int[].

Indexer kiểm tra chỉ số trước khi đọc hoặc ghi.

Phương thức TimSoChan trả về một đối tượng DaySoNguyen mới.

Công thức hoặc logic chính

Tổng các phần tử:

```
public int TinhTong()
{
   int tong = 0;

   foreach (int giaTri in mang)
   {
      tong += giaTri;
   }

   return tong;
```

```
}
```

Tìm giá trị lớn nhất:

```
public int TimLonNhat()
{
    if (mang.Length == 0)
    {
        throw new InvalidOperationException(
            "Mang dang rong."
        );
    }
    int lonNhat = mang[0];
    for (int i = 1; i < mang.Length; i++)
    {
        if (mang[i] > lonNhat)
        {
            lonNhat = mang[i];
        }
    }
    return lonNhat;
}
```

Kết quả minh họa

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai03_DaySoNguyen>dotnet run
CHUONG TRINH QUAN LY DAY SO NGUYEN

Nhap so luong phan tu: 2
Nhap phan tu [0]: 3
Nhap phan tu [1]: 6

DAY SO DA NHAP
3 6

CAC SO CHAN TRONG DAY
6

Phan tu dau tien: 3
Day so sau khi doi phan tu dau tien thanh 100:
100 6
```

Kết quả sau khi test

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02>cd Bai03_DaySoNguyenTest

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai03_DaySoNguyenTest>dotnet test
Restore complete (1,1s)
  Bai03_DaySoNguyen net10.0 succeeded (0,2s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai03_DaySoNguyen\bin\Debug\net10.0\Bai03_DaySoNguyen.dll
  Bai03_DaySoNguyenTest net10.0 succeeded (0,6s) → bin\Debug\net10.0\Bai03_DaySoNguyenTest.dll
[xUnit.net 00:00:00.01] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:00.70]   Discovering: Bai03_DaySoNguyenTest
[xUnit.net 00:00:00.76]   Discovered:  Bai03_DaySoNguyenTest
[xUnit.net 00:00:00.80]   Starting:    Bai03_DaySoNguyenTest
[xUnit.net 00:00:01.11]   Finished:    Bai03_DaySoNguyenTest
  Bai03_DaySoNguyenTest test net10.0 succeeded (3,5s)

Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 3,5s
Build succeeded in 6,3s
```

Giải thích cách làm

Mỗi chức năng là một phương thức riêng, dễ đọc và dễ viết Unit Test.

Tính tổng, tìm lớn nhất, nhỏ nhất và đếm phần tử đều có độ phức tạp O(n). Với mảng rỗng, những phép không có kết quả (như tìm số lớn nhất) phải xử lý riêng.

2.4 Mảng hai chiều

Phân tích đề bài

Xây dựng lớp quản lý ma trận số nguyên gồm nhiều dòng và cột, hỗ trợ:

- Nhập và xuất ma trận.
- Tính tổng các phần tử.
- Tìm phần tử lớn nhất.
- Tính tổng theo dòng hoặc cột.
- Tính tổng đường chéo chính nếu là ma trận vuông.
- Liệt kê các số nguyên tố có trong ma trận.
- Đọc hoặc thay đổi giá trị phần tử tại vị trí [i, j].

Hướng thiết kế và xử lý

Số dòng lấy bằng GetLength(0), số cột bằng GetLength(1).

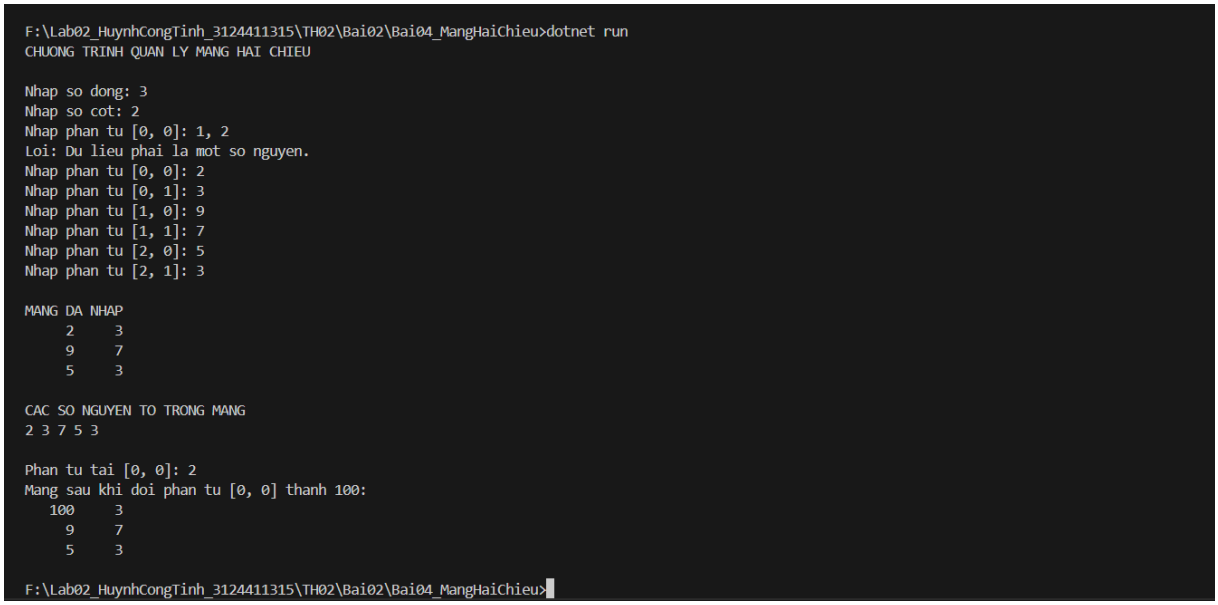
Số nhỏ hơn 2 không phải là số nguyên tố.

Chỉ cần thử các ước đến căn bậc hai của n.

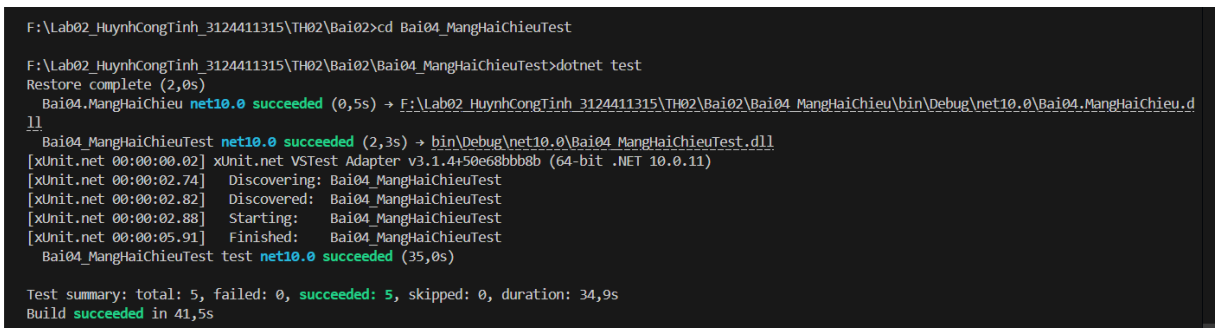
Công thức hoặc logic chính

LaSoNguyenTo(n): thu i tu 2 trong khi i <= n/i

Kết quả minh họa



Kết quả sau khi test:



Giải thích cách làm

Ma trận được duyệt bằng hai vòng lặp lồng nhau, vòng ngoài chạy theo dòng, vòng trong chạy theo cột.

Độ phức tạp của các thao tác duyệt toàn bộ ma trận là O(m×n)

Tổng đường chéo chính chỉ có nghĩa khi số dòng bằng số cột, nên phải kiểm tra ma trận vuông trước khi tính.

2.5 Đa thức

Phân tích đề bài

Xây dựng lớp DaThuc để biểu diễn:

P(x)=anx^n+⋯+a1x+a0

Mỗi đa thức gồm nhiều đối tượng DonThuc đã xây dựng ở bài 1.5.

Các chức năng chính:

- Thêm đơn thức.
- Thu gọn các đơn thức cùng số mũ.
- Sắp xếp theo số mũ.
- Tính giá trị đa thức.
- Tính đạo hàm.
- Cộng hai đa thức.
- Xuất đa thức.

Hướng thiết kế và xử lý

DaThuc tái sử dụng lớp DonThuc.

Vị trí i lưu đơn thức có số mũ i.

Phương thức TinhGiaTri cộng giá trị của từng đơn thức.

Công thức hoặc logic chính

P(x) = Tong(ai.x^i), i=0..n

Kết quả minh họa

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02>cd Bai05_DaThuc

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai05_DaThuc>dotnet run
CHUONG TRINH XU LY DA THUC

Nhap bac cua da thuc: 3
Nhap he so a0 cua x^0: 2
Nhap he so a1 cua x^1: 5
Nhap he so a2 cua x^2: 6
Nhap he so a3 cua x^3: 7

DA THUC DA NHAP
P(x) = 7x^3 + 6x^2 + 5x + 2
Nhap gia tri x: 2

KET QUA
P(x) = 7x^3 + 6x^2 + 5x + 2
P(2) = 92
Don thuc tai vi tri 0: 2
Don thuc bac cao nhat: 7x^3
```

Kết quả sau khi thực hiện

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02>cd Bai05_DaThucTesst
The system cannot find the path specified.

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02>cd Bai05_DaThucTest

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai05_DaThucTest>dotnet test
Restore complete (1,3s)
  Bai05_DonThuc net10.0 succeeded (0,2s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai05_DonThuc\bin\Debug\net10.0\Bai05_DonThuc.dll
  Bai05_DaThuc net10.0 succeeded (0,1s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai05_DaThuc\bin\Debug\net10.0\Bai05_DaThuc.dll
  Bai05_DaThucTest net10.0 succeeded (0,8s) → bin\Debug\net10.0\Bai05_DaThucTest.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:00.72]   Discovering: Bai05_DaThucTest
[xUnit.net 00:00:00.79]   Discovered:  Bai05_DaThucTest
[xUnit.net 00:00:00.83]   Starting:    Bai05_DaThucTest
[xUnit.net 00:00:01.25]   Finished:    Bai05_DaThucTest
  Bai05_DaThucTest test net10.0 succeeded (4,5s)

Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 4,4s
Build succeeded in 7,8s
```

Giải thích cách làm

DaThuc dùng lại lớp DonThuc. Khi thêm một đơn thức, chương trình tìm đơn thức cùng số mũ để cộng dồn hệ số, nên đa thức luôn ở dạng thu gọn.

Các đơn thức xếp theo số mũ giảm dần để in đúng dạng thường viết. Tính giá trị đa thức bằng cách gọi TinhGiaTri() của từng đơn thức rồi cộng lại.

2.6 Dãy phân số

Phân tích đề bài

Xây dựng lớp quản lý nhiều đối tượng PhanSo, hỗ trợ:

- Nhập và xuất dãy phân số.
- Tính tổng hoặc tích.
- Tìm phân số lớn nhất.
- Sắp xếp dãy phân số.

Hướng thiết kế và xử lý

DayPhanSo tái sử dụng lớp PhanSo và toán tử +.

Tổng ban đầu là 0/1.

Constructor và phương thức Add tạo bản sao để các dãy không dùng chung đối tượng.

Công thức hoặc logic chính

Tong = 0/1 + ps[0] + ps[1] + ...

Kết quả minh họa

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02>cd Bai06_DayPhanSo

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai06_DayPhanSo>dotnet run
CHUONG TRINH TINH TONG DAY PHAN SO

Nhap so luong phan so: 4

Nhap phan so thu 1:
Nhap tu so: 2
Nhap mau so: 3

Nhap phan so thu 2:
Nhap tu so: 4
Nhap mau so: 8

Nhap phan so thu 3:
Nhap tu so: 6
Nhap mau so: 9

Nhap phan so thu 4:
Nhap tu so: 6
Nhap mau so: 12

DAY PHAN SO DA NHAP
2/3 1/2 2/3 1/2

Tong day phan so: 7/3
Phan so dau tien: 2/3

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai06_DayPhanSo>cd ..
```

Kết quả sau khi test

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai06_DayPhanSoTest>dotnet test
Restore complete (1,1s)
  Bai04_PhanSo net10.0 succeeded (0,1s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai01\Bai04_PhanSo\bin\Debug\net10.0\Bai04_PhanSo.dll
  Bai06_DayPhanSo net10.0 succeeded (0,3s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai06_DayPhanSo\bin\Debug\net10.0\Bai06_DayPhanSo.dll
  Bai06_DayPhanSoTest net10.0 succeeded (0,8s) → bin\Debug\net10.0\Bai06_DayPhanSoTest.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:01.36]   Discovering: Bai06_DayPhanSoTest
[xUnit.net 00:00:01.45]   Discovered:  Bai06_DayPhanSoTest
[xUnit.net 00:00:01.49]   Starting:    Bai06_DayPhanSoTest
[xUnit.net 00:00:02.86]   Finished:    Bai06_DayPhanSoTest
  Bai06_DayPhanSoTest test net10.0 succeeded (7,2s)

Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 7,2s
Build succeeded in 10,6s
```

Giải thích cách làm

DayPhanSo dùng lại các phép toán của PhanSo, kết quả trung gian tự rút gọn.

Tìm phân số lớn nhất chỉ cần duyệt một lần, so sánh bằng nhân chéo. Sắp xếp có thể tự viết thuật toán hoặc dùng Array.Sort() nếu PhanSo cài đặt IComparable.

2.7 Tổng lương phòng ban

Phân tích đề bài

Quản lý phòng ban cùng danh sách nhân viên thuộc phòng ban đó, sau đó tính tổng tiền lương cần chi trả.

Quan hệ chính:

- Một phòng ban có nhiều nhân viên.
- Mỗi nhân viên có thông tin và cách tính lương.

Hướng thiết kế và xử lý

Dùng kiểu decimal cho số tiền.

Lớp NhanVien tính lương thực nhận của từng người.

Lớp PhongBan cộng lương của toàn bộ nhân viên.

Công thức hoặc logic chính

LuongThucNhan = MucLuong - SoNgayVang*100000

Kết quả minh họa

```
Nhap muc luong: 36536300
Nhap so ngay vang: 25

KET QUA
DANH SACH GOM 4 NHAN VIEN

Nhan vien thu 1:
Ho ten: Duong A
Muc luong: 15.000.000 VND
So ngay vang: 14
Luong thuc nhan: 13.600.000 VND

Nhan vien thu 2:
Ho ten: NGuyen B
Muc luong: 11.111.111 VND
So ngay vang: 3
Luong thuc nhan: 10.811.111 VND

Nhan vien thu 3:
Ho ten: Alexander
Muc luong: 2.222.222 VND
So ngay vang: 14
Luong thuc nhan: 822.222 VND

Nhan vien thu 4:
Ho ten: Ronaldo
Muc luong: 36.536.300 VND
So ngay vang: 25
Luong thuc nhan: 34.036.300 VND

Tong luong phong ban: 59.269.633 VND

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai07_TinhLuong>
```

Kết quả sau khi test

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai07_TinhLuongTest>dotnet test
Restore complete (1,5s)
  Bai07_TinhLuong net10.0 succeeded (0,4s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai07_TinhLuong\bin\Debug\net10.0\Bai07_TinhLuong.dll
  Bai07_TinhLuongTest net10.0 succeeded (1,5s) → bin\Debug\net10.0\Bai07_TinhLuongTest.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:02.04]   Discovering: Bai07_TinhLuongTest
[xUnit.net 00:00:02.14]   Discovered:  Bai07_TinhLuongTest
[xUnit.net 00:00:02.19]   Starting:    Bai07_TinhLuongTest
[xUnit.net 00:00:03.37]   Finished:    Bai07_TinhLuongTest
  Bai07_TinhLuongTest test net10.0 succeeded (23,3s)

Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 23,2s
Build succeeded in 27,8s

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai02\Bai07_TinhLuongTest>
```

Giải thích cách làm

PhongBan không tự tính lương thay nhân viên mà gọi TinhLuong() của từng người rồi cộng lại. Cách này hợp khi các loại nhân viên tính lương khác nhau.

Phần 3 Interface Delegate Kế thừa và Đa hình

3.1 Array.Sort và IComparable

Phân tích đề bài

Sắp xếp mảng đối tượng bằng Array.Sort(). Để C# xác định được thứ tự giữa hai đối tượng, lớp cần cài đặt IComparable<T>.

Hướng thiết kế và xử lý

Lớp SinhVien cài đặt IComparable<SinhVien>.

Phương thức CompareTo so sánh điểm, sau đó đến họ tên và mã sinh viên.

Array.Sort tự động gọi CompareTo.

Công thức hoặc logic chính

```
public class SinhVien : IComparable<SinhVien>
{
   public double DiemTrungBinh { get; set; }
   public int CompareTo(SinhVien? other)
   {
      if (other is null)
      {
         return 1;
      }
      return DiemTrungBinh.CompareTo(
         other.DiemTrungBinh
      );
   }
}
```

Quy ước của CompareTo():

- Nhỏ hơn 0: đối tượng hiện tại đứng trước.
- Bằng 0: hai đối tượng tương đương theo tiêu chí.
- Lớn hơn 0: đối tượng hiện tại đứng sau.

Kết quả minh họa

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03\Bai01_SinhVien>dotnet run
CHUONG TRINH SAP XEP SINH VIEN BANG ARRAY.SORT

Nhap so luong sinh vien: 3

Nhap sinh vien thu 1:
Nhap ma sinh vien: 145236987
Nhap ho ten: Nguyen Van A
Nhap diem trung binh: 8

Nhap sinh vien thu 2:
Nhap ma sinh vien: 123698547
Nhap ho ten: Nguyen Van B
Nhap diem trung binh: 7

Nhap sinh vien thu 3:
Nhap ma sinh vien: 147258963
Nhap ho ten: Ngo Mai A
Nhap diem trung binh: 5

DANH SACH TRUOC KHI SAP XEP
1. Ma SV: 145236987  | Ho ten: Nguyen Van A             | Diem TB: 8
2. Ma SV: 123698547  | Ho ten: Nguyen Van B             | Diem TB: 7
3. Ma SV: 147258963  | Ho ten: Ngo Mai A                | Diem TB: 5

DANH SACH SAU KHI SAP XEP TANG DAN THEO DIEM
1. Ma SV: 147258963  | Ho ten: Ngo Mai A                | Diem TB: 5
2. Ma SV: 123698547  | Ho ten: Nguyen Van B             | Diem TB: 7
3. Ma SV: 145236987  | Ho ten: Nguyen Van A             | Diem TB: 8

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03\Bai01_SinhVien>
```

Solution file opened: Lab02_HuynhCongTinh_3124411315.sln Indexing files... Connected to Discord CRLF C# Autocomplete Go Live Prettier

Kết quả sau khi test

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03>cd Bai01_SinhVienTest

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03\Bai01_SinhVienTest>dotnet test
Restore complete (1,1s)
  Bai01_SinhVien net10.0 succeeded (0,3s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03\Bai01_SinhVien\bin\Debug\net10.0\Bai01_SinhVien.dll
  Bai01_SinhVienTest net10.0 succeeded (2,5s) → bin\Debug\net10.0\Bai01_SinhVienTest.dll
[xUnit.net 00:00:00.17] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:03.39]   Discovering: Bai01_SinhVienTest
[xUnit.net 00:00:03.59]   Discovered:  Bai01_SinhVienTest
[xUnit.net 00:00:03.71]   Starting:    Bai01_SinhVienTest
[xUnit.net 00:00:04.91]   Finished:    Bai01_SinhVienTest
  Bai01_SinhVienTest test net10.0 succeeded (25,9s)

Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 25,8s
Build succeeded in 31,6s

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03\Bai01_SinhVienTest>
```

Giải thích cách làm

IComparable<T> quy định thứ tự mặc định của đối tượng. Cài đặt xong thì gọi thẳng Array.Sort(danhSach);

Muốn sắp giảm dần thì đảo dấu kết quả so sánh, hoặc sắp tăng dần rồi gọi Array.Reverse().

3.2 Sắp xếp tổng quát bằng interface

Phân tích đề bài

Tự xây dựng lại chức năng của Array.Sort bằng phương thức generic kết hợp bộ so sánh IComparer<T>.

Hướng thiết kế và xử lý

public interface IComparer<T>

```
{
   int Compare(T a, T b);
}
```

Thuật toán không cần biết T là số, chuỗi hay đối tượng; nó chỉ gọi Compare().

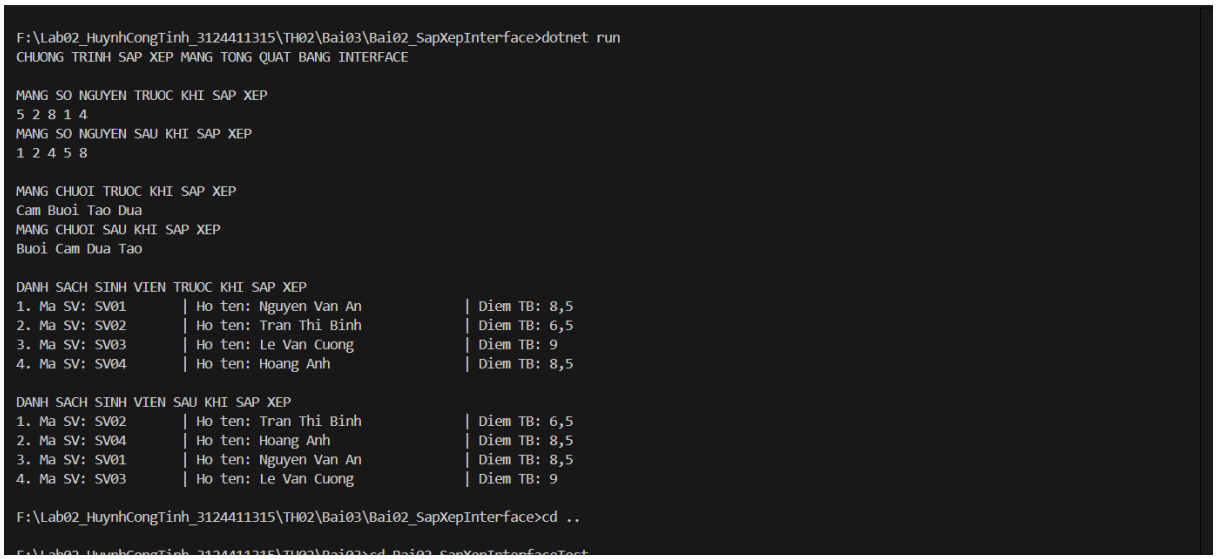
Công thức hoặc logic chính

```
public static void SapXep<T>(
   T[] mang,
   IComparer<T> boSoSanh)
{
   for (int i = 0; i < mang.Length - 1; i++)
   {
      int viTriNhoNhat = i;

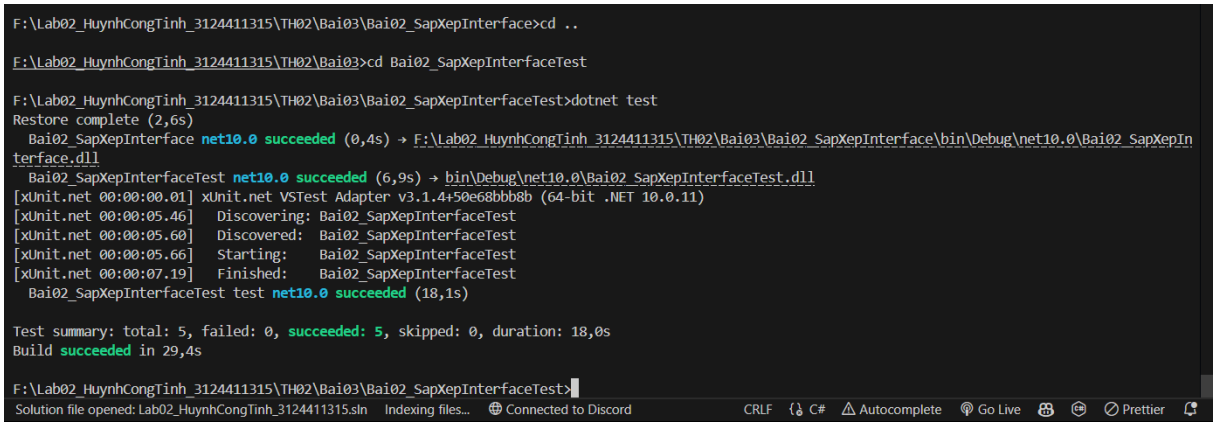

      for (int j = i + 1;
          j < mang.Length;
          j++)
      {
         if (boSoSanh.Compare(
                mang[j],
                mang[viTriNhoNhat]) < 0)
         {
            viTriNhoNhat = j;
         }
      }
```

```
        (mang[i], mang[viTriNhoNhat]) =
            (mang[viTriNhoNhat], mang[i]);
    }
}
```

Kết quả minh họa



Kết quả sau khi test



Giải thích cách làm

Thuật toán tổng quát không cần biết cấu trúc bên trong của T; nó chỉ gọi Compare() theo đúng hợp đồng của interface.

3.3 Sắp xếp tổng quát bằng delegate

Phân tích đề bài

Xây dựng thuật toán sắp xếp Generic dùng được cho nhiều kiểu dữ liệu, trong đó hàm so sánh được truyền vào bằng delegate thay vì tạo lớp cài đặt interface.

Hướng thiết kế và xử lý

Delegate SoSanh<T> đại diện cho hàm so sánh hai phần tử, thuật toán chỉ việc gọi soSanh(a, b).

Công thức hoặc logic chính

public delegate int SoSanh<T>(T a, T b);

Thuật toán:

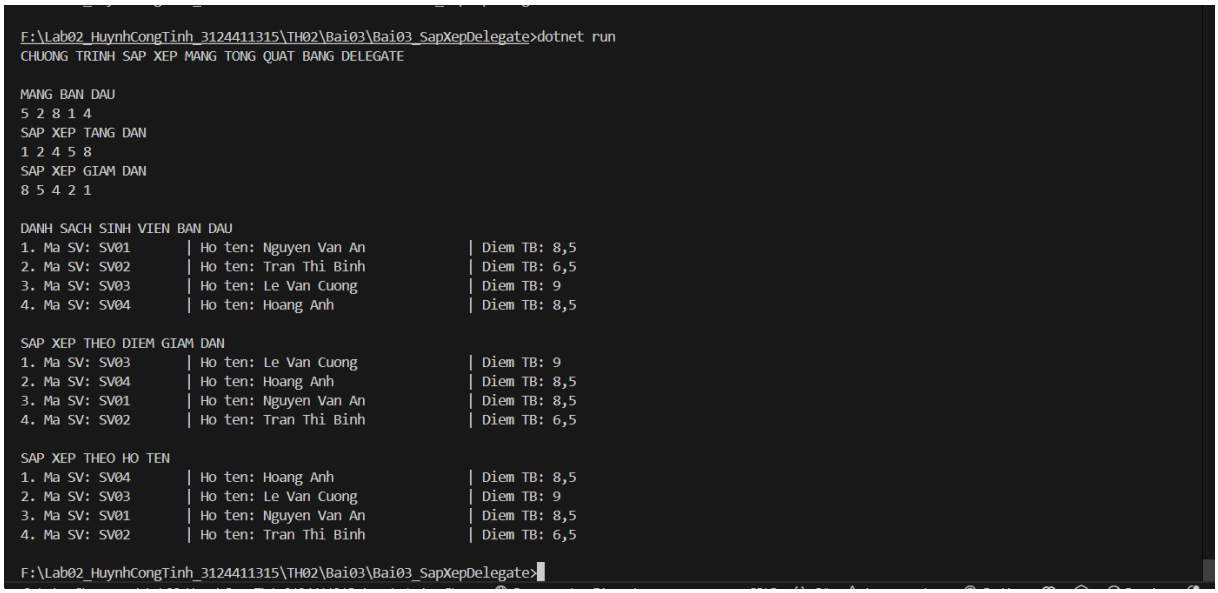
```
public static void SapXep<T>(
   T[] mang,
   SoSanh<T> soSanh)
{
   for (int i = 0; i < mang.Length - 1; i++)
   {
      for (int j = i + 1;
          j < mang.Length;
          j++)
      {
         if (soSanh(mang[i], mang[j]) > 0)
         {
            (mang[i], mang[j]) =
                (mang[j], mang[i]);
         }
      }
```

```
    }
}
```

Cách gọi:

```
SapXep(
    mang,
    (a, b) => a.CompareTo(b)
);
```

Kết quả minh họa



Giải thích cách làm

Delegate cho phép truyền một phương thức vào làm tham số, nên thuật toán không phụ thuộc kiểu dữ liệu hay tiêu chí so sánh cụ thể. Cùng một SapXep dùng được để sắp mảng số tăng hoặc giảm, và sắp danh sách sinh viên theo điểm hoặc họ tên.

So với interface, delegate gọn hơn khi tiêu chí đơn giản vì truyền thẳng được biểu thức lambda, không phải tạo thêm lớp.

3.4 ConsoleMenu và phương trình bậc hai

Phân tích đề bài

Xây dựng lớp ConsoleMenu tổng quát để hiển thị menu, nhận lựa chọn của người dùng và thông báo chức năng được chọn. Lớp được áp dụng cho bài toán giải phương trình bậc hai, có thể mở rộng thông qua sự kiện hoặc kế thừa.

Hướng thiết kế và xử lý

Lớp ConsoleMenu quản lý các mục chọn và phát sự kiện Choose khi người dùng chọn một chức năng.

Lớp PTBac2Console kế thừa ConsoleMenu và đăng ký xử lý sự kiện Choose.

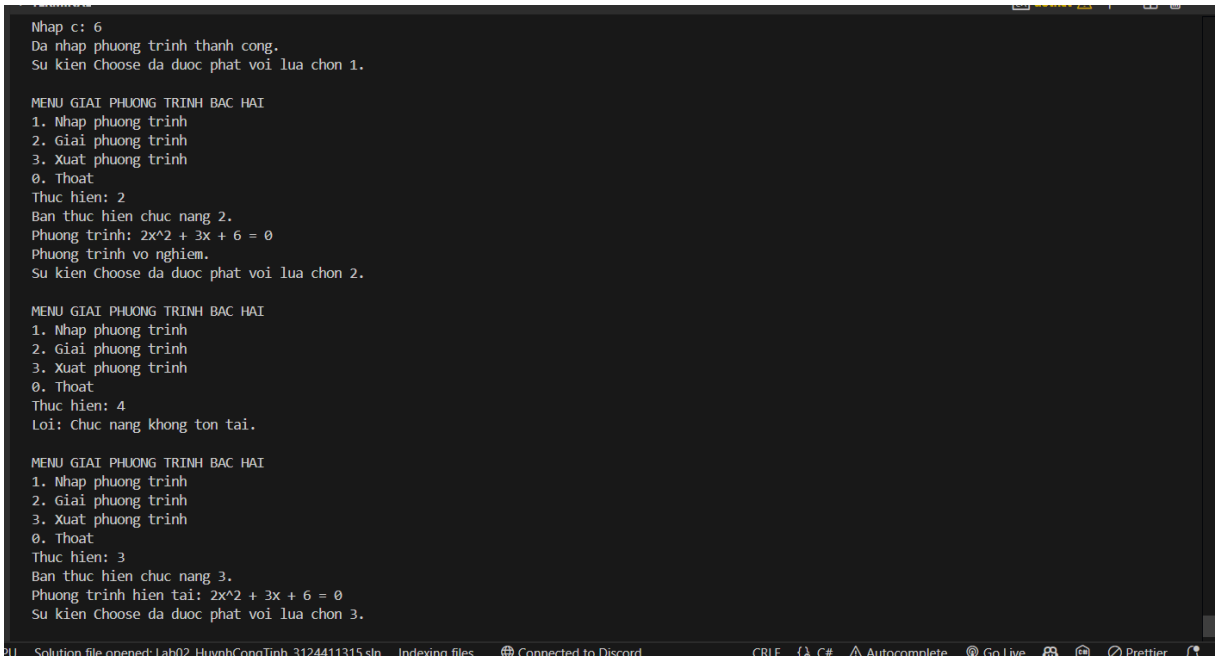
Lớp PhuongTrinhBacHai tách riêng phần nghiệp vụ giải phương trình để dễ kiểm thử.

Công thức hoặc logic chính

Vòng lặp menu: hiển thị menu → nhập lựa chọn x → nếu x khác 0 thì phát sự kiện Choose(x) → lặp lại cho đến khi x = 0.

Giải phương trình ax^2 + bx + c = 0: tính delta = b^2 - 4ac; delta < 0 thì vô nghiệm; delta = 0 thì có nghiệm kép; delta > 0 thì có hai nghiệm phân biệt.

Kết quả minh họa



Kết quả sau khi test

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03\Bai04_ConsoleMenu>cd ..

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03>cd Bai04_ConsoleMenuTest

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03\Ba  file:///F:/Lab02_HuynhCongTinh_3124411315/TH02/Bai03/Bai04_ConsoleMenu/bin/De
                                                 bug/net10.0 (ctrl + click)
Restore complete (1,4s)
  Bai04_ConsoleMenu net10.0 succeeded (1,0s) → F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03\Bai04_ConsoleMenu\bin\Debug\net10.0\Bai04_ConsoleMenu.dll
  Bai04_ConsoleMenuTest net10.0 succeeded (3,9s) → bin\Debug\net10.0\Bai04_ConsoleMenuTest.dll
[xUnit.net 00:00:00.03] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:06.10]   Discovering: Bai04_ConsoleMenuTest
[xUnit.net 00:00:06.58]   Discovered:  Bai04_ConsoleMenuTest
[xUnit.net 00:00:06.94]   Starting:    Bai04_ConsoleMenuTest
[xUnit.net 00:00:08.99]   Finished:    Bai04_ConsoleMenuTest
  Bai04_ConsoleMenuTest test net10.0 succeeded (27,7s)

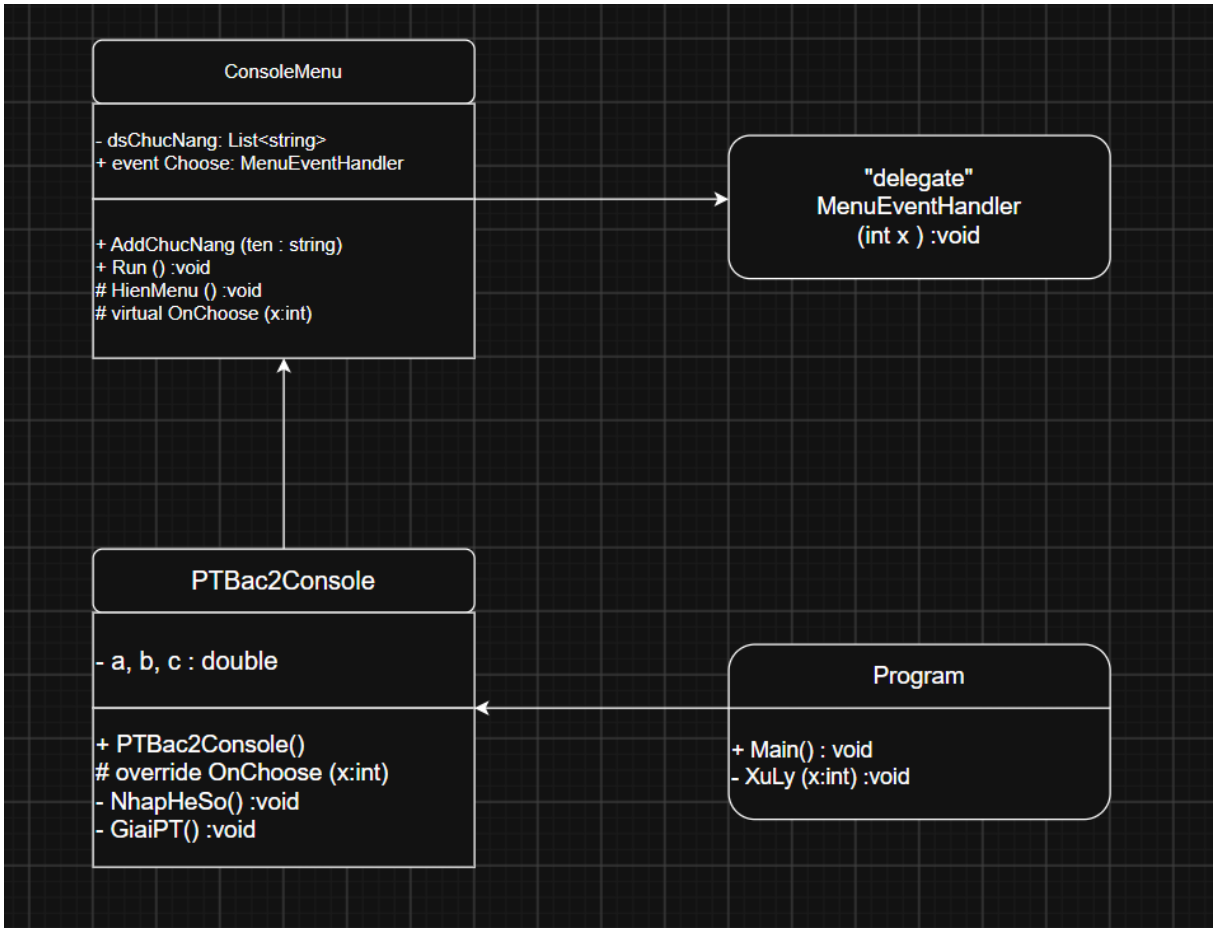
Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 27,5s
Build succeeded in 37,1s

F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai03\Bai04_ConsoleMenuTest>
```

Giải thích cách làm

ConsoleMenu chỉ lo phần menu: hiển thị, đọc lựa chọn và phát sự kiện Choose. Mỗi chức năng làm gì là việc của lớp con hoặc của nơi đăng ký sự kiện.

Có hai cách dùng: kế thừa như PTBac2Console, hoặc tạo đối tượng rồi đăng ký hàm xử lý bằng +=. Chọn số không có trong menu thì báo lỗi và hiện lại menu, chương trình không bị dừng.



Sơ đồ kế thừa ConsoleMenu

3.5 Tính lương bằng kế thừa và đa hình

Phân tích đề bài

Công ty có hai loại nhân viên là nhân viên kinh doanh và nhân viên sản xuất. Hai loại có chung mã nhân viên và họ tên nhưng công thức tính lương khác nhau.

Hướng thiết kế và xử lý

Lớp NhanVien là lớp trừu tượng, chứa thông tin chung.

NhanVienKinhDoanh nhận lương cơ bản cộng khoản tiền theo số hợp đồng đã ký.

NhanVienSanXuat tính lương theo số sản phẩm và thưởng thêm 5% khi làm trên 3000 sản phẩm.

CongTy lưu List<NhanVien> và gọi TinhLuong() theo cơ chế đa hình.

Công thức hoặc logic chính

KD: LuongCoBan + SoHopDong*500000
SX: SoSanPham*1000; neu >3000 thi cong 5%

Kết quả minh họa

```
F:\Lab02_HuynhCongTinh_3124411315\TH02\Bai04\Bai05_TinhLuongDaHinh>dotnet run
CHUONG TRINH TINH LUONG NHAN VIEN

DANH SACH GOM 3 NHAN VIEN

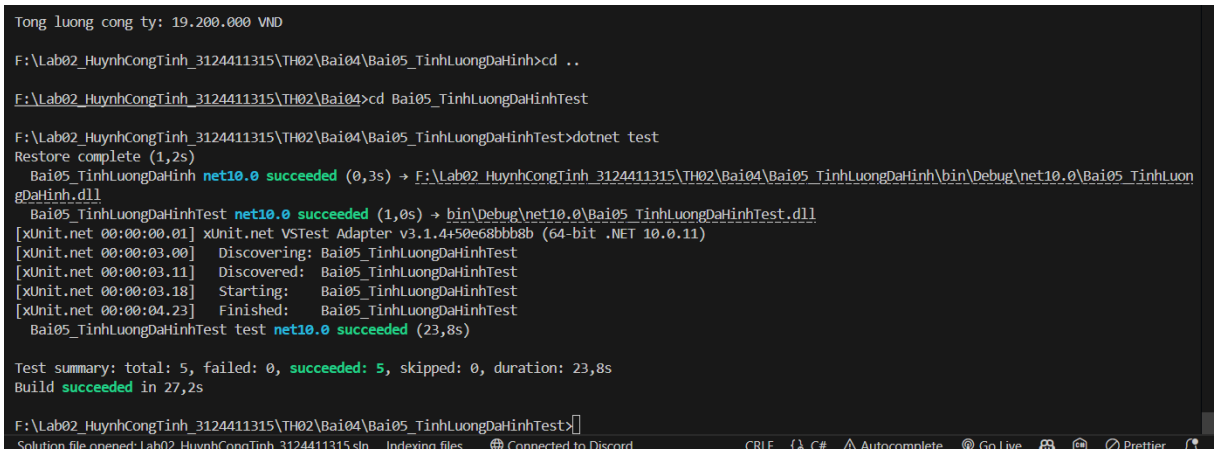
Nhan vien thu 1:
Ma nhan vien: KD01
Ho ten: Nguyen Van An
Loai: Nhan vien kinh doanh
Luong: 12.000.000 VND
Luong co ban: 10.000.000 VND
So hop dong: 4

Nhan vien thu 2:
Ma nhan vien: SX01
Ho ten: Tran Thi Binh
Loai: Nhan vien san xuat
Luong: 4.200.000 VND
So san pham: 4000

Nhan vien thu 3:
Ma nhan vien: SX02
Ho ten: Le Van Cuong
Loai: Nhan vien san xuat
Luong: 3.000.000 VND
So san pham: 3000

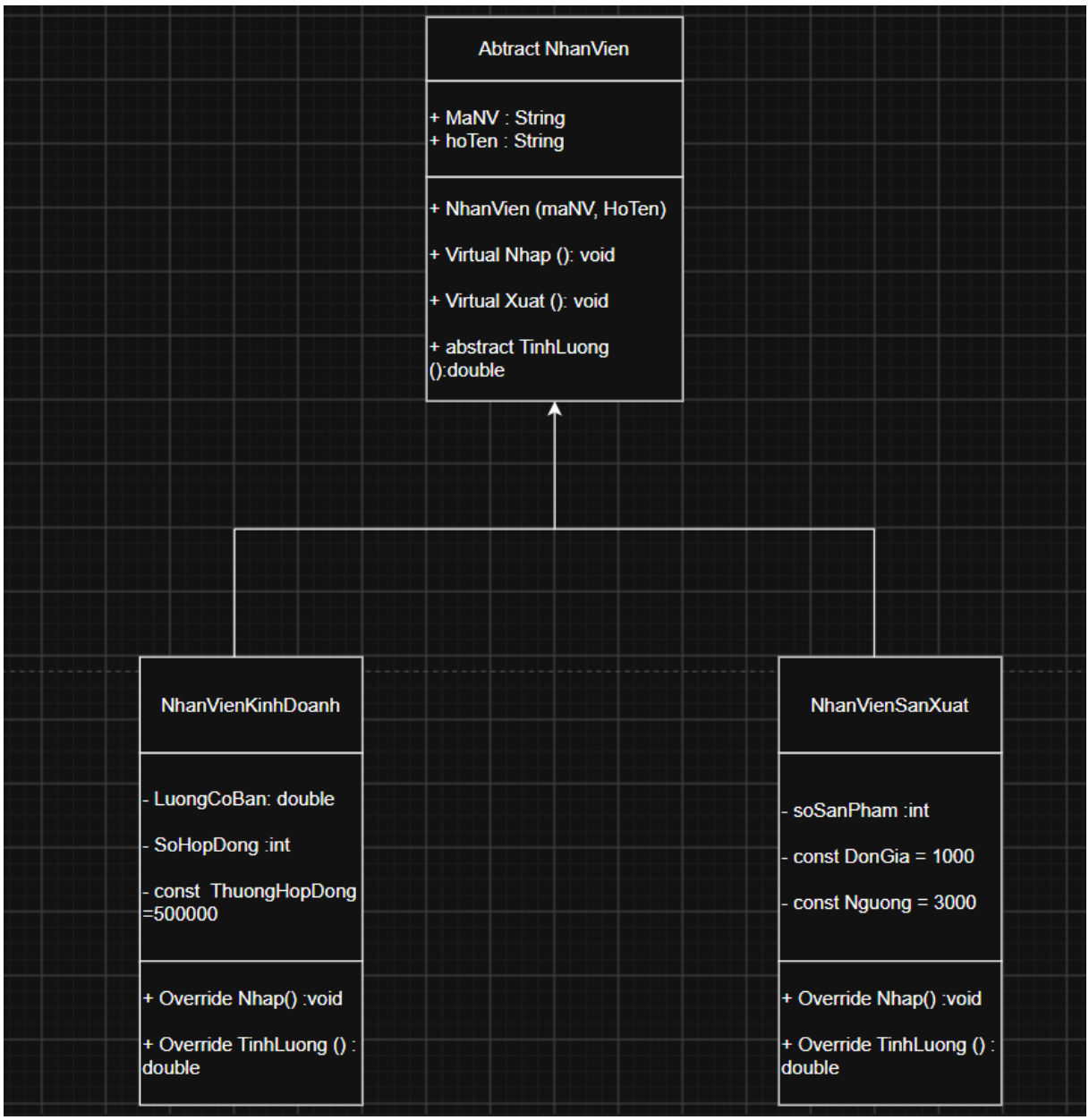
Tong luong cong ty: 19.200.000 VND
```

Kết quả sau khi test



Giải thích cách làm

Cùng gọi TinhLuong() nhưng mỗi loại nhân viên chạy đúng công thức của lớp mình.



Sơ đồ lớp tính lương nhân viên

3.6 Tính điểm thí sinh

Phân tích đề bài

Cuộc thi có hai đối tượng thí sinh là Chuyên và Siêu cúp. Cả hai cùng làm ba bài lập trình nhưng khác nhau về bài thi bổ sung và cách tính tổng điểm.

Hướng thiết kế và xử lý

Lớp ThiSinh là lớp trừu tượng.

ThiSinhChuyen cộng điểm thưởng tiếng Anh (từ 7 đến 8 điểm được cộng 1 điểm, từ 9 đến 10 điểm được cộng 2 điểm).

ThiSinhSieuCup cộng trực tiếp điểm CSDL.

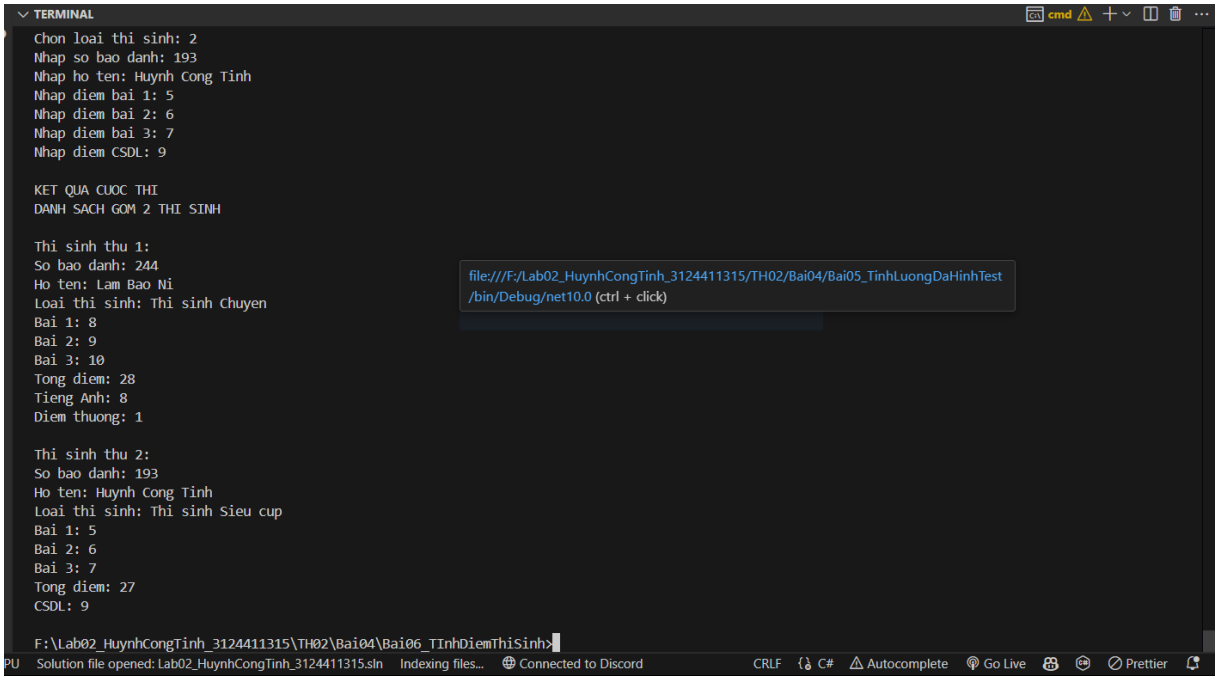
Lớp CuocThi lưu List<ThiSinh>.

Công thức hoặc logic chính

Chuyen = Bai1+Bai2+Bai3+Thuong

SieuCup = Bai1+Bai2+Bai3+CSDL

Kết quả minh họa



Kết quả sau khi test

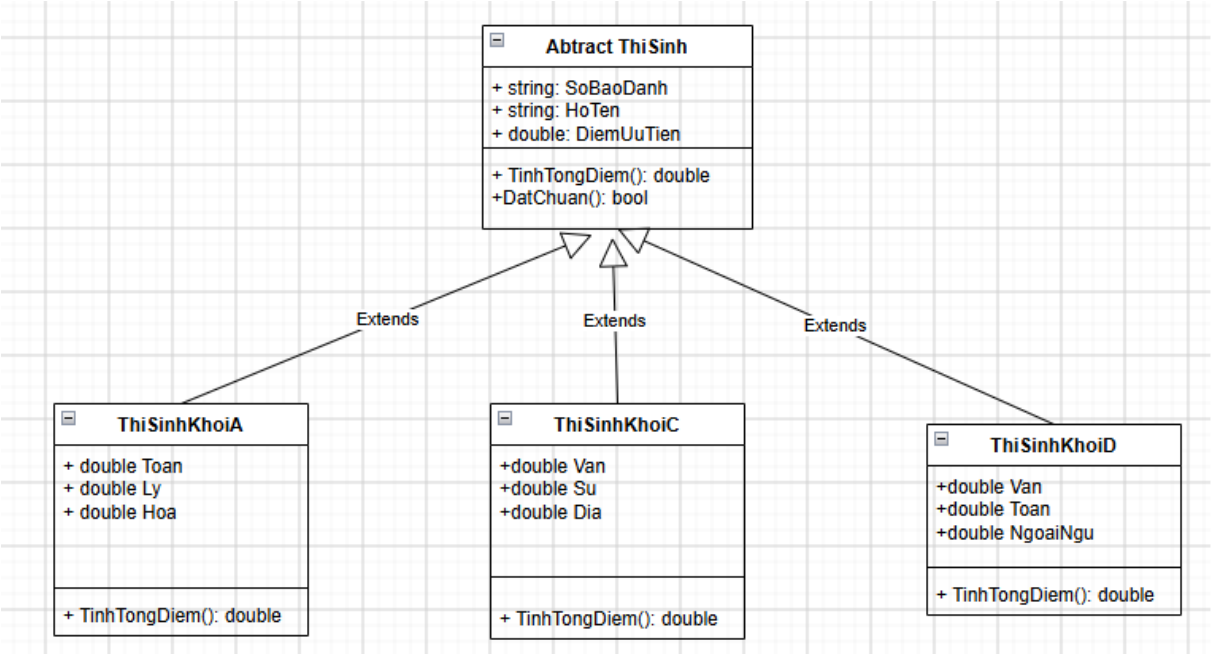


Giải thích cách làm

Lớp ThiSinh chứa thông tin chung như số báo danh, họ tên và điểm của ba bài lập trình. Phương thức TinhTongDiem() được khai báo trừu tượng vì mỗi đối tượng thí sinh có bài thi bổ sung và cách tính điểm khác nhau.

Mỗi lớp con ghi đè TinhTongDiem() theo công thức riêng. List<ThiSinh> chứa được cả hai loại thí sinh, duyệt danh sách và gọi TinhTongDiem() là mỗi thí sinh tự tính theo đúng loại của mình.

Muốn thêm loại thí sinh mới chỉ cần tạo lớp kế thừa ThiSinh và cài đặt TinhTongDiem(), không phải sửa các lớp đã có.



Sơ đồ lớp thí sinh

Phần 4 Kết luận và hướng dẫn chạy Unit Test

Kiểm thử bằng xUnit

Mỗi bài sử dụng năm Unit Test, tập trung vào kết quả đúng, giá trị biên và dữ liệu không hợp lệ. Project test chỉ tham chiếu một chiều đến project chính. Có thể chạy một project test bằng lệnh:

dotnet test .\TenProjectTest\TenProjectTest.csproj

dotnet test --logger "console;verbosity=detailed"

Kết luận

Qua bài thực hành này, em đã làm quen với các nội dung chính của lập trình hướng đối tượng trong C#: đóng gói bằng field và property, constructor, indexer, nạp chồng toán tử, rồi đến interface, delegate, event, kế thừa và đa hình.

Các lớp nghiệp vụ được tách khỏi Program nên dùng lại và kiểm thử riêng được. Dữ liệu được kiểm tra ngay trong property và constructor, còn ArrayPoint, PersonList, DaThuc, DayPhanSo đều tận dụng lại các lớp viết từ trước.

Mỗi bài có năm Unit Test xUnit cho trường hợp đúng, giá trị biên và dữ liệu sai, trong đó có các lỗi hay gặp như mẫu số bằng 0, số mũ âm, chỉ số ngoài phạm vi hay mảng rỗng.

Ở các bài cuối, interface và delegate cho phép tách cách so sánh khỏi thuật toán sắp xếp, còn kế thừa và đa hình cho phép thêm loại nhân viên hay loại thí sinh mới mà không sửa code cũ.

Bài còn hạn chế là vẫn dùng ArrayList ở một số chỗ. Sau này nên chuyển sang collection generic và bổ sung thêm test.